Project Phases, deliverables, and Guidelines

Phase 1.1: Selecting the appropriate data structures that should be used to store different lists in the project.
Phase 1.2: Implementing and testing the selected data structures to make sure they work as required to proceed to phase 2.
Phase 2: Implementing the algorithms related to the project logic to simulate different scenarios and generate the output file containing system operation statistics.

Project Features and the weight of each feature will be uploaded

Phase 1

This phase is concerned with selecting the appropriate data structures **for the whole project,** not just for Phase 1.

Phase 1.1: Choose the data structures for each list in the project.

You are allowed to use the following **data structures ONLY:**

- Stack, Queue, Pri-queue
- or any class derived from them having the needed additional functions

Deliverables: You should deliver a report in the following format.

Phase 1.2: Implement & Test the data structures.

ALL data structures in the project MUST make use of the Stack, Queue, or Priority Queue classes **given in the labs** or classes derived from them.

NOT ALLOWED: **Global variables, Use of friendship, C++ STL.**

SHARE, MOVE, Don't Copy

Don't allocate the same object (order, chef, etc) more than once in the program. Hence, all lists should be **lists of pointers** so that when an object moves from one list to another, you just make the new list point to it, not to take a new copy of the object.

Phase 1.2 Code

You should finalize the following in phase1.2:

1. ***Full data members*** *of classes (See Appendix A below)*
2. ***Full implementation*** *for all data structures that you will use to represent different lists of orders, chefs,.. etc.* ***Must Use DS code given in the lab or classes derived from them.***
3. ***UI class & UI-related function*** *Add Print and count to ALL DS, Overload << operator on order* and other classes as needed. Add print member inside order and other*

classes as needed

4. ***Simple RANDOM Simulator function for Phase 1***. The main purpose of this function is to test different lists and make sure all operations on them are working properly.

Random Simulation Function Details. The function should:

1. Perform any needed initializations
2. Randomly generate at least 500 pending orders that are randomly distributed on all pending orders lists. **You can generate these orders with the help of AI.**
3. **At each time step follow the following algorithm:**
   Note: at any step if there is no order/chef/table/scooter to pick move to the next part of the algorithm
   - 3.1. Repeat the following steps **30 times**
     - Randomly pick the top order from one of the pending lists
     - Pick a random chef for this order and move that order to the Cooking list(s).
   - 3.2. With probability 75%, Pick a random order from the cooking list(s) and distribute it randomly on the ready orders lists. Repeat this 15 times.
   - 3.3. Repeat the following steps **10 times**
     - Randomly pick the top order from one of the ready lists
     - If the order is OT, move to finish list
     - If the order is OV, pick a random scooter. If OD, pick a random table. The move the order to In-service list
   - 3.4. Generate a random order ID and call cancel function of the "pending OVC" list to get that order (if found) and place it in the cancelled orders list.
   - 3.5. Generate a random order ID and call cancel function of the "ready OVC" list to get that order (if found) and place it in the cancelled orders list.
   - 3.6. Generate a random order ID and call cancel function of the "cooking OV" list to get that order (if found) and place it in the cancelled orders list and release its chef.
   - 3.7. With probability 25%, pick the top order from the In-service list and place it in the finish list.
     - If it is OV, place its scooter in the Scooter_Back list.
     - If it is OD, place its table in the tables list.
   - 3.8. With probability 50%, pick a scooter from the Scooter_Back list and randomly make it available again or make it go to maintenance.
   - 3.9. With probability 50%, pick a scooter from maintenance list to free scooters list.
   - 3.10. Print info of all lists (except actions) as described in the "Program Interface".
4. End the simulation when ALL generated orders are in finish or cancel lists.

Notes:

- *NO actual order assignment logic is required in Phase 1.* However, all the lists of the project should be fully operating in phase1.
- *No input or output file are required in phase1.*
- *Silent mode is not required at this phase*

Phase1.2 Video

Record a short video (less than 10 minutes) showing a demo for phase1. you should:

- Give a **very** brief explanation for the types of data structures lists you have chosen for each list in the project. (max 2 min)
- Show how the first random pending orders are generated.
- Run the code and explain how orders are moved from one list to another.

Phase 1.2 Deliverables:

Each team is required to submit:

1. A text file named ***ID.txt*** containing team members' names, IDs, and emails.
2. **Phase1.2 code** [Do not include executable files].
3. **Phase1 Video (upload to Gdrive, Give access to everyone, share link)**

Phase 2

In this phase, you should extend the code of phase 1 to build the full application, read the input file and produce the final output file, and support the different operation modes.

Load must be distributed equally between team members.

Phase 2 Deliverables:

Each team is required to submit the following:

- A text file named ***ID.txt*** containing team members' names, IDs, and emails.
- **Team Workload document**. Showing load of each member.
- **Final project code** [Do not include executable files].
- **6 comprehensive sample input files** (test cases) and ***their output files***.
  File1: 50 actions, File2:100 actions, and so on till File6:300 actions. Sample input files must cover non-trivial scenarios.
- **One of the sample input files** designed for clear illustration for the cancellation scenarios.
- **Test_description.txt**, In this file, write which input files test which scenarios and specify the input file testing most scenarios.

Project Evaluation

Project Grade: 40% (phase 1.1, phase 1.2) and 60% (Phase 2)

- ☐ **Work Load:** Each member must be responsible for writing some project modules and answer some questions showing understanding of the logic and implementation of his part. The workload of team members must be equal.
- ☐ **Individual grading for EACH item (% is from Phase 2 Grade) :**
  - **[60%]** Code understanding and quality
  - **[20%]** Code run
  - **[20%]** Integrating work with members who finished or nearly finished.
- ☐ You should **inform the TAs** before the deadline **with sufficient time (some weeks before)** if any problems happen between team members.
- ☐ If one or more members don't do their work, the other members will NOT be affected **as long as** (1) they inform us beforehand about the problem, (2) all other parts are integrated, AND (3) *their part can be easily tested to see the output*.

Penalties:

The cheating penalty (giving or taking) any project part from any source is ZERO grade in the project AND also MINUS FIVE (-5) from total classwork grade, so it is better to deliver it incomplete than cheated.

It is totally your responsibility to keep your code away from others.

Appendix A - Guidelines for project Code

Action classes:

There are two types of actions; Request and Cancel actions. You should create a base class called "***action***" for common actions data. This is an abstract class with a pure virtual function "***Act***". The logic of "***Act***" function depends on the action type.

For each action type, there should be a class derived from ***action*** class. Each derived class should store data related to its type. Also each of them should override function **Act** as follows:

1- ***RequestAction::act***:Creates a new order and adds it to the appropriate list through calling member function(s) in class Restaurant.

2- ***CancelAction::act:***Cancels the order if found

Class Restaurant has a queue of ***action*** pointers to store all actions loaded from the file at system startup. At each timestep, the code loops on the actions queue to dequeue and execute all actions that should take place at the current timestep.

order, chef, scooter, table Classes

To hold all info about order and other restaurant objects in the project

Restaurant Class

This class is the maestro class that manages the system. It should hold appropriate **lists** of orders and chefs, ..etc

It should have member functions to:

- ☐ At program startup, open the input file and load all actions to the actions list
- ☐ At each timestep, (**and among other things**) ⇒ This is a Phase 2 function.
  - ☐ Check actions list and for actions with time = current timestep, execute them
  - ☐ Check the maintenance list to see if a scooter finishes its fix to move to avail list
  - ☐ Check if a table will be partially or fully available.
  - ☐ Move finished orders to "Finish" list and release the assigned scooter/table
  - ☐ Assign pending orders to chefs
  - ☐ Assign ready orders to scooter/table or give to customer (OT orders)
  - ☐ Collect statistics that are needed to create output file
  - ☐ Call UI class functions to print details on the output screen
- ☐ Produce the output file at the end of simulation ⇒ This is a Phase 2 function.

UI Class

It should be responsible for reading any inputs from the user and printing any information on the **console**. It contains the input and output functions that you should use to

show the status of the system at the end of each timestep.

How to print the output screen described in the "Program Interface" section.

- In each data structure (DS) you will use (e.g. queue, priority queue, etc.) , you should add a ***print*** member function that loops on its data and prints it formatted as shown in the Program Interface section. You are also allowed to add a ***counter*** data member and ***GetCount()*** in different DS to be able to print their count. These are the ONLY allowed edits in the stack, queue, and priority queue classes of the lab code.
- You should also ***overload << operator*** for ***order**** (and any other classes as needed) to make it print their info. You may use friendship here.

The Role of UI class when printing the o/p screen.

Then in UI class, add a member function that takes as parameters pointers/references to these lists. This function prints the output screen in the required format. When it needs to print the lists passed to it, it should just call the print function of each list.

In addition, the UI class should have functions to be called to read the interface mode, input and output file **names.**

The UI class is **NOT** responsible for reading/writing the input/output files